

Week 4 – Background Subtraction (Image Segmentation)

Purpose of this Week's Lab:

The purpose of this week's lab is to continue developing practical skills in image processing using Python. In particular, we will focus on:

- Background subtraction for separating foreground objects from a scene
- Binary image segmentation using thresholding
- Morphological image operations (dilation and erosion)
- Applying masks to images
- Writing reusable Python functions
- Refactoring code for cleaner program design

By the end of this lab, you will have implemented a reusable background subtraction pipeline that can process multiple images automatically.

Preparation

Make sure the following Python packages are installed:

```
pip install opencv-python numpy
```

Import libraries in Python:

```
import cv2
import numpy as np
import os
```

Review of Previous Week

Open your homework from last week on **background subtraction**.

Discuss with your tutor:

- Were you able to highlight the dingo in the image?
- What challenges did you observe?
- Which parts of the foreground were not well segmented?

Last week you also rescaled image intensities so pixel values ranged between **0 and 255**. In the example code, I manually coded the min-max normalisation to show you the computation process, but in Python this can be done using:

```
img = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
```

Compare this with your own solution.

Background Subtraction

OK, so this week, let's improve segmentation quality using some additional processing steps.

Recall that background subtraction detects moving objects by comparing: an input image with a background image. Pixels that differ significantly are considered foreground.

Step 1 — Load Images

```
input_img = cv2.imread("DINGO3_Frame0.jpg")
background_img = cv2.imread("DINGO3_Background.jpg")
```

Step 2 — Convert to Grayscale

This is because many processing steps work better on grayscale images.

```
gray_input = cv2.cvtColor(input_img, cv2.COLOR_BGR2GRAY)
gray_background = cv2.cvtColor(background_img, cv2.COLOR_BGR2GRAY)
```

Step 3 — Compute Difference Image

```
diff_img = cv2.absdiff(gray_input, gray_background)
```

This highlights pixels that changed between frames.

Step 4 — Thresholding

This converts a grayscale image into a binary image.

```
threshold_value = 40
_, binary_mask = cv2.threshold(
    diff_img,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)
```

Here, you need to experiment with different threshold values. What threshold best separates the dingo from the background?

Step 5 — Morphological Operations (new)

After thresholding, segmentation is usually imperfect. Small holes and noise remain. Two important operations help refine results:

Dilation → expands white regions

Erosion → shrinks white regions

Now let's create the structuring element, a kernel:

```
kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE,
    (5, 5)
)
```

Now, try different shapes and sizes.

Perform dilation, then observe how foreground regions grow:

```
dilated = cv2.dilate(binary_mask, kernel)
```

Perform erosion, then observe how noise is removed:

```
eroded = cv2.erode(dilated, kernel)
```

Now, let's do the combined operation (a.k.a., closing) and apply dilation and erosion together

```
clean_mask = cv2.morphologyEx(
    binary_mask,
    cv2.MORPH_CLOSE,
    kernel
)
```

Experiment with kernel size and shape. How do these parameters affect segmentation quality?

Step 6 — Applying the Binary Mask

Now, in the original image, we keep only pixels belonging to the foreground object, using this `clean_mask` we generated.

The concept is that for each pixel, let's apply the mask:

Mask = 0 → set pixel to 0 (remove)

Mask = 1 → keep original pixel

We can do this through two approaches:

Method 1 — Elementwise Multiplication (Recommended)

```
mask_3ch = cv2.cvtColor(clean_mask, cv2.COLOR_GRAY2BGR) / 255.0
```

```
cut_out_img = (input_img * mask_3ch).astype(np.uint8)
```

Method 2 — Using Loops (not recommended, why? ---- slow, inefficient)

```
result = input_img.copy()
```

```
for y in range(clean_mask.shape[0]):
    for x in range(clean_mask.shape[1]):
        if clean_mask[y, x] == 0:
            result[y, x] = 0
```

Compare performance between the two approaches.

Step 7 — Refactoring Code into a Function

Now, we can make our code more efficient. Instead of rewriting code for every image, we can create a reusable function. Then, we can call this function when we need to process an image or many images. Below, I defined a function called, `compute_background_subtracted_img`, it has two inputs (an input image and a background image), and will return two outputs: the binary mask and the cut-out image. Inside the function, it includes all the processing steps we have implemented above.

```
def compute_background_subtracted_img(input_img, background_img):
```

```
    # Convert to gray
```

```

gray_input = cv2.cvtColor(input_img, cv2.COLOR_BGR2GRAY)
gray_background = cv2.cvtColor(background_img, cv2.COLOR_BGR2GRAY)

# Compute difference
diff_img = cv2.absdiff(gray_input, gray_background)
# Normalisation
diff_img = cv2.normalize(diff_img, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)

# Thresholding
_, binary_mask = cv2.threshold(diff_img, 40, 255,
                              cv2.THRESH_BINARY)

# Create structuring element
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))

# Perform dilation and erosion
clean_mask = cv2.morphologyEx(
    binary_mask,
    cv2.MORPH_CLOSE,
    kernel
)

# Apply the binary mask
mask_3ch = cv2.cvtColor(clean_mask,
                        cv2.COLOR_GRAY2BGR) / 255.0

cut_out_img = (input_img * mask_3ch).astype(np.uint8)

return clean_mask, cut_out_img

```

Step 8 — Now we can use this function for processing an image or multiple frames

Let's say we have two frames with dingo, and of course the background image. We can reuse the function to segment the dingo from the two frames.

OK, now we load two images and with background image, and use our function for segmentation

```

background_img = cv2.imread("DINGO3_Background.jpg")
dingo_img1 = cv2.imread("DINGO3_Frame0.jpg")
dingo_img2 = cv2.imread("DINGO3_Frame477.jpg")

dingo_images = [dingo_img1, dingo_img2]

for i, img in enumerate(dingo_images):

    mask, result = compute_background_subtracted_img(
        img,
        background_img
    )

plt.figure(figsize=(12, 6))

```

```
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title(f"Image{i+1}")
plt.axis('off')
```

```
plt.subplot(1, 3, 2)
plt.imshow(mask)
plt.title(f"Mask{i+1}")
plt.axis('off')
```

```
plt.subplot(1, 3, 3)
plt.imshow(result)
plt.title(f"Result{i+1}")
plt.axis('off')
```

Step 9 — Homework: Processing Multiple Frames

If you did not finish all tasks during the lab, complete them before the next class.

In particular:

1. Finalise your refactored implementation.
2. Ensure your program:
 - Uses a reusable function
 - Processes multiple images automatically
3. Experiment with:
 - Different thresholds
 - Different structuring element sizes
4. Briefly document which parameters produced the best segmentation results.

Place all images into one folder, write code to loop over all images in the folder, and run your code.

Observe:

- Does the same threshold still work?
- Does segmentation quality change?